

EMPLOYEE DATABASE MANAGEMENT USING MYSQL (DDL & CONSTRAINTS)

Project Overview

This project focuses on designing and managing an Employee Database using MySQL Data Definition Language (DDL) commands. The database stores information about employees, departments, and office locations while maintaining data integrity through the use of appropriate constraints and relationships.

The project demonstrates practical database design concepts, schema modification, table management, and constraint implementation commonly used in real-world database administration.

Objectives

- Design a relational database for employee management.
- Create and manage database objects using MySQL DDL commands.
- Establish relationships between multiple tables using foreign keys.
- Apply database constraints to maintain data accuracy and consistency.
- Perform schema modifications using ALTER statements.
- Practice table and database management operations.

Database Design

The database consists of three related tables.

1. Departments_Info

Stores department details.

Table: **department_info**

Columns:

<u>department_id</u>	int PK
department_name	varchar(100)

Column	Description
department_id	Unique department identifier (Primary Key)
department_name	Name of the department

2. Locations

Stores company location details.

Table: **locations**

Columns:

<u>location_id</u>	int PK
location_name	varchar(30)

Column	Description
location_id	Auto-incrementing location identifier (Primary Key)
location_name	Name of the company location

3. Employees

Stores employee information.

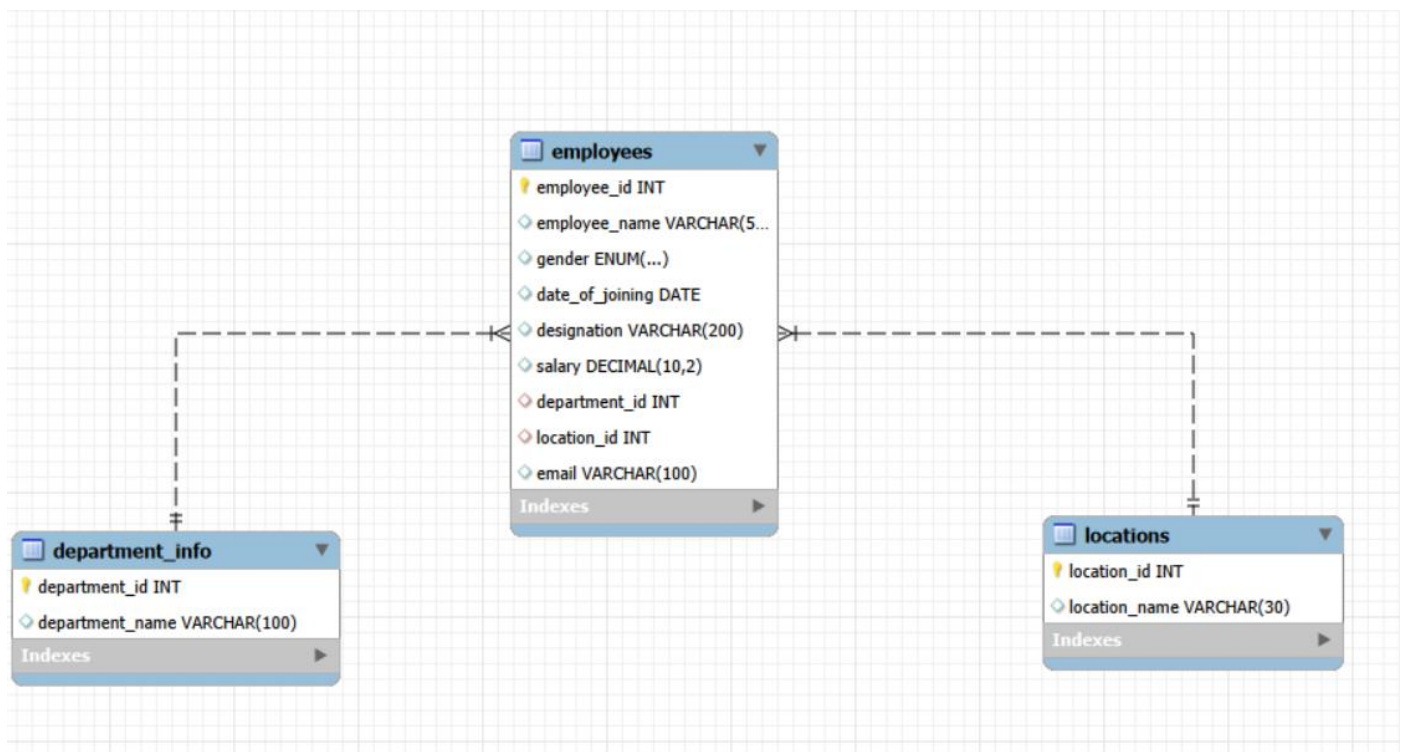
Table: **employees**

Columns:

<u>employee_id</u>	int PK
employee_name	varchar(50)
gender	enum('female','male')
date_of_joining	date
designation	varchar(200)
salary	decimal(10,2)
department_id	int
location_id	int
email	varchar(100)

Column	Description
employee_id	Unique employee identifier (Primary Key)
employee_name	Employee's full name
gender	Employee gender (M/F)
age	Employee age
hire_date	Date of joining
designation	Employee job title
salary	Employee salary
email	Employee email address
department_id	Foreign Key referencing Departments_Info
location_id	Foreign Key referencing Locations

Entity Relationship



- One Department can have multiple Employees.
- One Location can have multiple Employees.
- Every Employee belongs to one Department.
- Every Employee works at one Location.

DDL Operations Performed

Database Creation

- Created the Employee database.

Query: CREATE DATABASE Employee_db;

- Selected the database for further operations.

Query: USE employee_db;

Table Creation

Created the following tables:

- Departments
- Location
- Employees

Implemented:

- Primary Keys
- Foreign Keys
- Appropriate data types

Query:

--- create table

```
create table departments(  
    department_id int primary key,  
    department_name varchar(100)  
);
```

--- create table

```
create table location(  
    location_id int primary key,  
    location_name varchar(30)  
);
```

--- create table

```
create table employees(  
    employee_id int primary key,  
    employee_name varchar(50),  
    gender enum('female' , 'male'),  
    age int,  
    hire_date date,  
    designation varchar(100),  
    salary decimal(10,2),  
    department_id int,  
    location_id int,
```

Foreign Key (department_id) references departments(department_id),

Foreign Key (location_id) references location(location_id)

);

Table Alteration

Performed various schema modifications including:

- Added Email column.

Query:

```
alter table employees  
add email varchar(100);
```

- Modify the data type of the "designation" column in the Employees table

Query:

```
alter table employees  
modify designation varchar(200);
```

- Drop the “age” column from the Employees table

Query:

```
alter table employees  
drop column age;
```

- Rename the “hire_date” column to date_of_joining

Query:

```
alter table employees  
rename column hire_date to date_of_joining;
```

Table Renaming

- Rename the "Departments" table to "Departments_Info"

Query:

```
rename table departments to department_info;
```

- Rename the "Location" table to "Locations"

Query:

```
rename table location to locations;
```

Table Truncation

Removed all employee records using the TRUNCATE command while preserving the table structure.

Query:

```
truncate table employees;
```

Database and Table Deletion

Performed:

- DROP TABLE Employees

Query: drop table employees;

- DROP DATABASE Employee

Query: drop database employee_db;

Action Output

Output				
Action Output				
#	Time	Action	Message	
✓ 1	00:04:43	DROP DATABASE `employee_db`	3 row(s) affected	
✓ 2	00:04:52	CREATE DATABASE Employee_db	1 row(s) affected	
✓ 3	00:05:17	USE employee_db	0 row(s) affected	
✓ 4	00:05:17	create table departments(department_id int primary key, department_name varchar(100))	0 row(s) affected	
✓ 5	00:05:17	create table location(location_id int primary key, location_name varchar(30))	0 row(s) affected	
✓ 6	00:05:17	create table employees(employee_id int primary key, employee_name varchar(50), gender enum(fe...	0 row(s) affected	
✓ 7	00:05:17	alter table employees add email varchar(100)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 8	00:05:17	alter table employees modify designation varchar(200)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 9	00:05:17	alter table employees drop column age	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 10	00:05:17	alter table employees rename column hire_date to date_of_joining	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	
✓ 11	00:05:17	rename table departments to department_info	0 row(s) affected	
✓ 12	00:05:18	rename table location to locations	0 row(s) affected	
✓ 13	00:24:48	truncate table employees	0 row(s) affected	
✓ 14	00:24:48	drop table employees	0 row(s) affected	
✓ 15	00:24:48	drop database employee_db	2 row(s) affected	

Constraints Implemented

Departments Table

- Primary Key on Department_ID
- Department_Name set as NOT NULL
- Department_Name made UNIQUE

Table: **departments**

Columns:

<u>department_id</u>	int PK
department_name	varchar(100)

Query:

```
create table departments(  
    department_id int primary key unique,  
    department_name varchar(100) not null unique  
);
```

Locations Table

- Location_ID configured as PRIMARY KEY with AUTO_INCREMENT
- Location_Name set as NOT NULL
- Location_Name made UNIQUE

Table: **location**

Columns:

<u>location_id</u>	int AI PK
location_name	varchar(30)

Query:

```
create table location(  
    location_id int primary key unique auto_increment,  
    location_name varchar(30) not null unique  
);
```

Employees Table

Implemented the following constraints:

- Employee_ID as PRIMARY KEY
- Employee_Name as NOT NULL
- Gender restricted using ENUM('M','F')
- Age validated using CHECK (Age >= 18)
- Hire_Date assigned CURRENT_DATE by default
- Department_ID as FOREIGN KEY
- Location_ID as FOREIGN KEY

Table: employees

Columns:

employee_id	int PK
employee_name	varchar(50)
gender	enum('M','F')
age	int
hire_date	date
designation	varchar(100)
salary	decimal(10,2)
department_id	int
location_id	int

Query:

```
create table employees(  
    employee_id int primary key unique,  
    employee_name varchar(50) not null,
```

```

gender enum('M','F'),
age int check (age >=18),
hire_date date default(current_date()),
designation varchar(100),
salary decimal(10,2),
department_id int,
location_id int,

Foreign Key (department_id) references departments(department_id),
Foreign Key (location_id) references location(location_id)

);

```

Action Output

Output				
Action Output				
#	Time	Action	Message	
✓ 1	00:25:30	CREATE DATABASE Employee_db	1 row(s) affected	
✓ 2	00:25:56	USE employee_db	0 row(s) affected	
✓ 3	00:25:56	create table departments(department_id int primary key unique, department_name varchar(100) not null unique)	0 row(s) affected	
✓ 4	00:25:56	create table location(location_id int primary key unique auto_increment, location_name varchar(30) not null unique)	0 row(s) affected	
✓ 5	00:25:56	create table employees(employee_id int primary key unique, employee_name varchar(50) not null, gender enum('M','F'), age int check (age >=18), hire_date date default(current_date(...	0 row(s) affected	

Data Integrity Features

The database ensures:

- Unique identification of every employee.
- No duplicate departments or locations.
- Mandatory employee names.
- Valid gender values only.
- Employees must be at least 18 years old.
- Automatic hire date assignment.
- Referential integrity between employees, departments, and locations.

SQL Concepts Demonstrated

- CREATE DATABASE
- CREATE TABLE
- ALTER TABLE
- MODIFY COLUMN
- ADD COLUMN
- DROP COLUMN
- RENAME TABLE
- RENAME COLUMN
- TRUNCATE TABLE
- DROP TABLE
- DROP DATABASE
- PRIMARY KEY
- FOREIGN KEY
- AUTO_INCREMENT
- NOT NULL
- UNIQUE
- CHECK Constraint
- DEFAULT Constraint
- ENUM Data Type

Learning Outcomes

Through this project, the following database concepts were practiced:

- Relational database design
- Schema creation and modification
- Constraint implementation
- Primary and foreign key relationships
- Data integrity enforcement

- Database object management using MySQL DDL commands
- Best practices for maintaining consistent and reliable data

Conclusion

This project successfully demonstrates the creation and management of a relational Employee Database using MySQL. By implementing DDL commands, constraints, and relationships, the database ensures accuracy, consistency, and integrity while following standard database design principles used in enterprise applications.